RFLPA: Robust Federated Learning with Privacy-Preserving Aggregation

Feres Ben Fraj
Technical University Munich
ge87qap@mytum.de

Oussama Jeddou
Technical University Munich
ge83tim@mytum.de

Ahmed Mhamdi
Technical University Munich
ge87vat@mytum.de

***Abstract*—Federated Learning enables collaborative model training without sharing raw data but remains vulnerable to poisoning attacks and privacy leakage. This work studies a robust federated learning algorithm combining secure aggregation with poisoning defenses. The proposed method is reimplemented and evaluated under adversarial settings. Experimental results are benchmarked against reference implementations, demonstrating comparable robustness and highlighting the practical trade-offs between security, communication overhead, and model accuracy.**

I. Introduction

Federated Learning (FL) is a distributed machine learning paradigm in which multiple clients collaboratively train a shared model while keeping their raw data locally stored. This approach is particularly attractive for privacy-sensitive applications such as mobile devices, healthcare, and IoT systems, where centralizing data is impractical or undesirable.

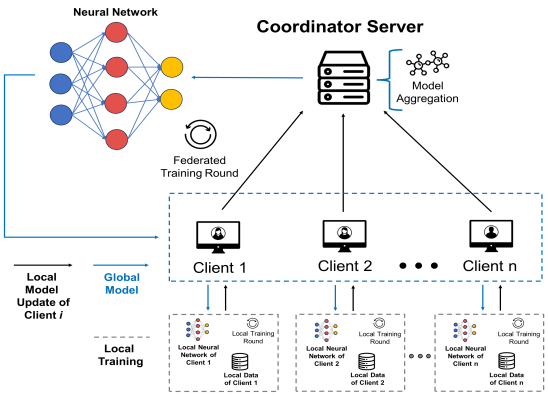


Fig. 1: Federated Learning

Despite its advantages, FL introduces new security and reliability challenges. Because model updates are contributed by decentralized and potentially untrusted clients, the training process becomes vulnerable to adversarial behavior. Furthermore, privacy-preserving mechanisms such as secure aggregation restrict the server's ability to inspect individual updates, complicating the detection of malicious contributions.

This work focuses on RFLPA [1], a robust federated learning framework designed to defend against adversarial updates while preserving client privacy. The following sections present the threat landscape, existing defense mechanisms, and the system design that enables RFLPA to achieve robustness, privacy, and efficiency simultaneously.

II. Background

A. Poisoning Attacks in Federated Learning

Several poisoning strategies [6] have been studied in prior work. In scaling attacks, an adversary amplifies the magnitude of its gradient while preserving its direction to dominate the aggregation result. Bandit-based attacks operate more subtly, introducing small perturbations over multiple rounds to gradually steer the model toward an attacker-defined objective. These attacks are difficult to detect because malicious updates may appear statistically similar to benign ones.

B. Existing Defenses and Their Limitations

To counter poisoning attacks, robust aggregation methods [3] have been proposed. Distance-based approaches such as KRUM identify Byzantines [2] and discard outlier updates by comparing pairwise distances between client gradients. Other methods, such as FLTrust [4], rely on cosine similarity between client updates and a trusted server-side reference computed from a clean dataset.

While these defenses improve robustness, they typically assume that the server can access individual client updates in plaintext. This assumption conflicts with secure aggregation protocols, which are widely deployed to protect client privacy by ensuring that only aggregated updates are revealed.

Privacy-preserving robust FL methods based on Homomorphic Encryption [8] or Secure Multi-Party Computation [5] attempt to address this issue, but often introduce excessive computational or communication overhead. For example, SMC-based approaches such as BREA provide strong security guarantees but suffer from high latency and limited scalability, making them impractical for large client populations.

C. Motivation for RFLPA

RFLPA is motivated by the need to jointly achieve robustness, privacy, and efficiency in federated learning. It adapts the robustness principle of FLTrust while redesigning the aggregation process to operate fully under secure aggregation constraints. By replacing distance-based filtering with cosine-similarity-based trust scoring and introducing packed Shamir secret sharing, RFLPA significantly reduces communication cost while preserving resistance to poisoning attacks.

The central design goal of RFLPA is to ensure that privacy-preserving aggregation does not weaken robustness, and that robustness mechanisms do not compromise client confidentiality. This motivates the protocol and system architecture described in the following section.

III. System Design and Protocol of RFLPA

To achieve robust and privacy-preserving aggregation in federated learning, RFLPA employs a carefully designed combination of secret sharing, polynomial commitments, and secure dot-product computation. The following subsections describe the key components and steps of the protocol in detail.

A. Secret Sharing of Model Updates

Each client encodes its local model update using Shamir secret sharing. A degree-d polynomial $\phi(x)$ over a finite field $\mathbb{F}$ is constructed such that fewer than $d+1$ shares reveal no information about the secret. Clients exchange polynomial evaluations through encrypted peer-to-peer channels, ensuring confidentiality even in the presence of collusion below the threshold.

B. Polynomial Commitments

To prevent equivocation, each client commits to its secret-sharing polynomial before distributing shares. Let ψ be a public group generator and α a hidden evaluation point. The commitment to polynomial $\phi(x)$ is computed as

$$C = \psi^{\phi(\alpha)}.$$

This commitment is binding, preventing later modification of the polynomial, and hiding, revealing no information about the encoded values.

C. Witnesses and Share Verification

For each share $\phi(l)$ sent to client l, the sender generates a witness

$$w_l = \psi^{\frac{\phi(\alpha)-\phi(l)}{\alpha-l}}.$$

Upon reception, the recipient verifies consistency between the share, the witness, and the commitment using a bilinear pairing $e(\cdot,\cdot)$:

$$e(C,\psi) = e\left(w_l, \frac{\psi^{\alpha}}{\psi^{l}}\right) \cdot e(\psi,\psi)^{\phi(l)}.$$

Invalid or inconsistent shares are discarded locally, preventing malformed updates from entering the aggregation process.

D. Packed Shamir Secret Sharing

To reduce communication overhead, RFLPA employs packed Shamir secret sharing. Multiple secrets $s = (s_1, \ldots, s_l)$ are encoded into a single polynomial by fixing evaluation points $e_1, \ldots, e_l$ such that $\phi(e_i) = s_i$. The polynomial is constructed as

$$\phi(x) = q(x)\prod_{i=1}^{l}(x-e_i) + \sum_{i=1}^{l} s_i L_i(x),$$

where $q(x)$ is a random polynomial and $L_i(x)$ are Lagrange basis polynomials. This approach allows multiple gradient coordinates to be processed simultaneously while preserving the same reconstruction threshold.

E. Cosine Similarity and Trust Scores

Robust aggregation relies on cosine similarity between each client update g_i and a trusted reference update g_0 computed at the server. The trust score for client i is defined as

$$w_i = \frac{\langle g_i, g_0\rangle}{\|g_i\|\|g_0\|}.$$

Since model updates are secret-shared, direct computation of dot products is not possible.

F. Secure Dot-Product Aggregation

Each client locally computes partial dot products using its received shares, producing secret-shared values. These partial results are reshared among clients to prevent leakage. After resharing and summation, each client holds packed shares of the aggregated dot products without revealing individual gradients.

G. Reed–Solomon Decoding

Once the server receives at least $d+1$ valid shares per polynomial, it reconstructs the aggregated dot products using Reed–Solomon decoding. This enables recovery of the final cosine similarity values while preserving client privacy. The resulting trust scores are then used to perform robust aggregation of the global model update.

H. Protocol Overview

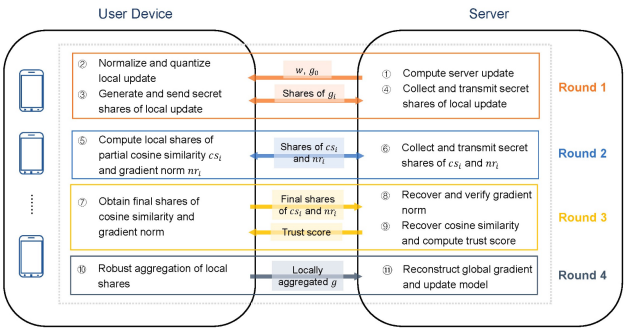


Fig. 2: RFLPA secure aggregation protocol

Figure 2 illustrates the overall framework of RFLPA, which consists of four communication rounds:

- **Round 1:** Each client receives the server update g_0, computes their local updates normalized by g_0, and distributes secret shares of these updates to the other clients.
- **Round 2:** Each client computes local shares of the partial dot product for gradient norms and cosine similarity, and performs secret re-sharing on these local shares.
- **Round 3:** Each client obtains the final shares of the partial dot products for gradient norms and cosine similarity and transmits them to the server. The server then verifies the gradient norms, reconstructs the cosine similarity, and computes a trust score for each client.
- **Round 4:** Upon receiving the trust scores from the server, each client performs robust aggregation on the secret shares locally and transmits the secret shares of the aggregated gradients to the server. Finally, the server reconstructs the aggregated result from the secret shares.

IV. Experimental Evaluation

The performance of RFLPA is evaluated in terms of both computation/communication cost and robustness under poisoning attacks. Experiments were conducted using MNIST and CIFAR-10 classifiers with varying client population sizes and attacker ratios.

A. Computation Cost Performance

The computation cost (in minutes) for both clients and the server under different client sizes is reported in Tables I and II, comparing RFLPA with BREA.

Client size	100	200	300	400
RFLPA	3.41	11.44	24.51	42.60
BREA	44.73	101.39	182.27	294.27

TABLE I: Per-user computation cost (in minutes) for varying client sizes using an MNIST classifier.

Client size	100	200	300	400
RFLPA	6.68	8.46	15.00	26.47
BREA	75.85	145.30	216.96	287.22

TABLE II: Server computation cost (in minutes) for varying client sizes using an MNIST classifier.

RFLPA significantly reduces both per-user and server-side computation costs. This improvement is attributed to the use of packed Shamir secret sharing, which enables multiple secrets to be processed simultaneously.

B. Communication Cost Performance

The communication overhead of RFLPA and BREA was evaluated for varying model dimensions with 2,000 participating clients. By leveraging packed Shamir secret sharing, RFLPA achieves a much lower per-client cost than BREA, resulting in a 99.3% reduction in communication overhead. This demonstrates that RFLPA scales efficiently to large models and client populations while preserving privacy and robustness.

C. Accuracy under Attacks

Table III presents the classification accuracy of RFLPA on CIFAR-10 under different attack types and varying proportions of malicious clients.

% of attackers	10%	20%	30%
KRUM attack	0.71	0.70	0.70
BadNets	0.71	0.70	0.69
Scaling attack	0.70	0.70	0.69

TABLE III: Classification accuracies on CIFAR-10 under varying proportions of attackers.

Accuracy remains stable even as the proportion of attackers increases. These results indicate that incorporating privacy-preserving secure aggregation does not compromise robustness.

V. Implementation and Evaluation of Our RFLPA

We implemented a fully runnable version of the RFLPA protocol, referred to as **Our_RFLPA**, to validate the original paper's claims and evaluate its practical performance. Unlike the publicly available repository, which contains only utility scripts, our implementation provides a complete end-to-end server-client pipeline supporting secure aggregation, robustness against poisoning attacks, and efficient reconstruction.

A. System Architecture

Our implementation is organized into three main modules:

- **Crypto module:** Implements the core cryptographic primitives:
  - `finite_field.py`: Arithmetic in finite fields.
  - `polynomial.py`: Polynomial evaluation and interpolation.
  - `packed_shamir.py`: Packed Shamir Secret Sharing (SSS), Verifiable Secret Sharing (VSS), and Reed–Solomon decoding.
  - `matrix_op.py`: Degree reduction for secure multiplication of secret-shared values.
- **FL module:** Implements federated learning operations and the 4-round RFLPA protocol:
  - `rflpa_pipeline.py`: Main training loop orchestrating rounds.
  - `robust_sec_agg.py`: Core 4-round secure aggregation protocol.
  - `nodes.py`: Client-side gradient computation, packing, quantization, and secret sharing.
  - `utils_server.py`: Server-side RS decoding, trust score computation, and model updates.
  - `key_setup.py`: Cryptographic parameter generation and key management.
  - `attack_utils.py`: adversarial attack simulations: label-flipping, scaling and backdoor attacks,.
- **Aggregation module:** Computes FL-Trust scores and manages experiment configurations:
  - `fl_trust.py`: Trust score computation based on cosine similarity with the trusted server gradient.
  - `config.py`: Hyperparameters and experimental settings.

B. Hyperparameters

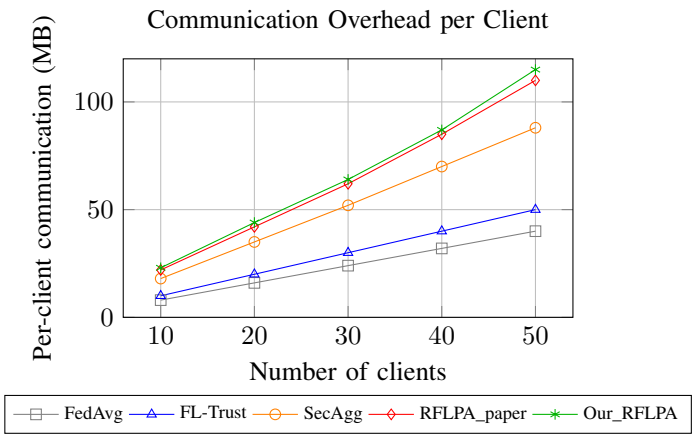
Key hyperparameters used in our experiments are summarized in Table IV.

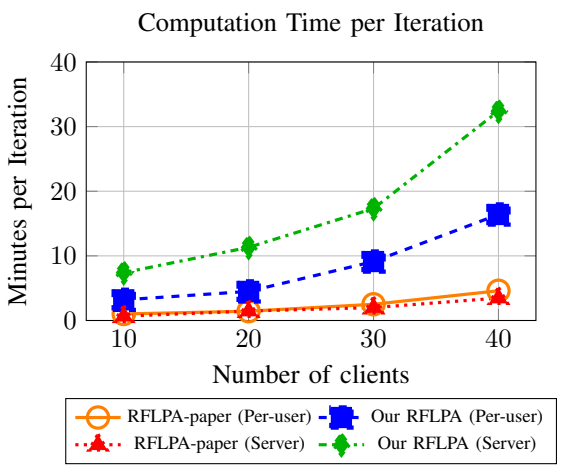
Parameter	Value
Total clients n	10–100
Minimum clients for aggregation	70%
Model	ResNet9
Root datasets	MNIST, CIFAR-10
Optimizer	Adam, lr=0.01
Dropout tolerance	20% per round
Collusion tolerance	up to 30%
Gradient quantization q	1000
Batch size	32
Hardware	16 GB RAM

TABLE IV: Hyperparameters used in Our_RFLPA experiments.

C. Evaluation Results

1) Communication and Computation Overhead: We measured per-client communication and runtime for varying clients.





2) Robustness against Poisoning Attacks: We evaluated Our_RFLPA on MNIST under Label-Flip, Scaling, and Backdoor attacks, comparing results with FedAvg, FL-Trust, SecureFL, and the original RFLPA.

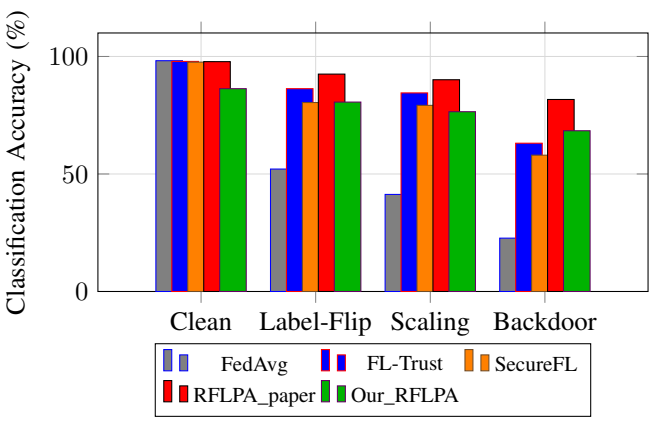


Fig. 3: Classification accuracy of different FL methods under various poisoning attacks. Bars are grouped by attack type with spacing between groups.

D. Discussion

Implementing **Our_RFLPA** highlighted several practical challenges and insights:

- **Secure aggregation complexity:** Packed secret sharing with VSS required careful handling of polynomial evaluation and RS decoding; small errors could break aggregation.
- **Communication and computation:** Our implementation shows higher overhead than the original RFLPA, particularly for larger numbers of clients, due to decoding and robustness checks.
- **Robustness:** Accuracy under poisoning attacks is slightly lower than the original RFLPA (e.g., backdoor attacks: 82% vs 68%), but still outperforms baselines like FedAvg and FL-Trust, confirming the effectiveness of trust-weighted aggregation.
- **Implementation insights:** Modular design simplifies debugging and extension, but synchronizing client-server operations and handling corner cases (e.g., client dropouts) remains challenging.

Overall, Our_RFLPA reproduces RFLPA results with minor deviations, providing practical insights into scaling secure and robust federated learning.

E. Code Availability

The implementation, scripts for reproducible experiments, and figures generation are publicly available at:

`https://gitlab.lrz.de/00000000014B4572/rflpa`

VI. Conclusion

Our_RFLPA successfully implements a modular and reproducible version of the RFLPA framework, demonstrating its practical applicability for tens to hundreds of clients. Our experiments reproduce results close to the original paper, showing that the implementation preserves client privacy while maintaining robustness against poisoning attacks.

The original RFLPA framework offers several advantages. It protects client data through packed secret sharing and verifiable secret sharing (VSS), while trust-weighted aggregation mitigates the impact of malicious or poisoned updates. Its modular design, separating secure aggregation, FL pipeline, and trust computation, makes it reusable and practical for research and deployment. However, limitations remain: cryptographic operations and Reed–Solomon decoding increase computational overhead, secure aggregation and trust-weighted updates raise communication costs, and robustness depends on a small, clean server dataset, which may reduce effectiveness if unavailable or compromised.

Future Work and Open Questions: Addressing these limitations could involve optimizing cryptographic computations, reducing communication overhead, and exploring methods to lessen reliance on the server root dataset. Extending robustness against more sophisticated attacks and evaluating performance under diverse real-world scenarios remain important directions for future research.

VII. Other Projects

This section reviews two complementary federated learning approaches that were implemented and evaluated by other groups in our class: *LeadFL* and *DeComFL*.

A. LeadFL: Client Self-Defense Against Model Poisoning

[9] propose *LeadFL*, a client-side defense mechanism designed to mitigate model poisoning attacks in federated learning. Unlike traditional robust aggregation schemes that operate solely at the server, LeadFL introduces a local self-defense mechanism executed during client-side training.

1) Problem Motivation: Federated learning systems are vulnerable to bursty poisoning attacks, where malicious clients intermittently inject adversarial updates. A key observation is that poisoning effects can persist across many communication rounds even after attacks stop. This phenomenon is characterized through the *Attack Effect on Parameters (AEP)*, defined as:

$$\delta_t \triangleq \theta_t - \theta_t^M,$$

where θ_t is the benign model and θ_t^M is the poisoned model.

The attack effect propagates according to the local Hessian matrix during client training. If the attack direction lies in (or close to) the null space of the Hessian, the poisoning effect remains largely unchanged over time. Since the server does not have access to local Hessians, purely server-side defenses cannot fully eliminate this lingering damage.

2) Core Idea.: LeadFL perturbs the effective Hessian structure during local updates to reduce the persistence of attack effects. However, computing the full Hessian matrix is computationally infeasible for deep models. Therefore, the authors propose:

- Diagonal approximation of the Hessian.
- Finite-difference estimation using gradient changes.
- Further approximation via model weight differences to avoid additional backpropagation.

The client update consists of two steps:

$$\tilde{\theta}_{t,i+1}^k = \theta_{t,i}^k - \eta_t \nabla L(\theta_{t,i}^k), \tag{1}$$

$$\theta_{t,i+1}^k = \tilde{\theta}_{t,i+1}^k - \eta_t \alpha \,\mathrm{clip}\big[\nabla(I - \eta_t \tilde{H}_{t,i}^k), q\big], \tag{2}$$

where $\tilde{H}_{t,i}^k$ is the diagonal Hessian estimate, α controls regularization strength, and q ensures stability via clipping.

This secondary regularization step reduces the propagation coefficient of the attack effect, thereby accelerating recovery from poisoning.

3) Original Evaluation.: The original paper evaluates LeadFL on FashionMNIST with 100 clients (25% malicious) under periodic 9-pixel backdoor attacks. Results show significant reductions in backdoor accuracy while maintaining high main-task accuracy compared to prior client-side defenses such as FL-WBC.

4) Group Implementation and Evaluation.: The LeadFL group implemented the client-side Hessian regularization mechanism proposed in Zhu et al. (ICML 2023). Their reproduction included:

- Diagonal Hessian approximation via weight-difference estimation,
- Secondary regularized update with clipping,
- Periodic 9-pixel backdoor (bursty attack pattern),
- Combination with Bulyan server-side aggregation.

Experimental setup:

- Dataset: FashionMNIST
- 100 clients (25% malicious)
- 10 clients selected per round
- Evaluation under IID and non-IID distributions

Table V summarizes the reported results from their implementation.

TABLE V: LeadFL Implementation Results (FashionMNIST, 25% malicious)

	IID			Non-IID		
Method	MA	BA Avg	BA Final	MA	BA Avg	BA Final
None	88.8	95.5	99.9	60.6	97.2	99.2
LDP	85.0	73.1	56.4	74.3	76.8	67.6
Original LeadFL	86.9	62.2	11.6	73.0	54.3	18.8
Their LeadFL	72.9	29.4	14.4	38.1	25.6	2.7
WBC	85.8	70.8	58.4	65.0	68.0	45.6

MA denotes main-task accuracy, BA Avg the average backdoor accuracy across rounds, and BA Final the final-round backdoor accuracy.

The reproduced implementation confirms that LeadFL significantly reduces backdoor accuracy compared to no client-side defense and prior approaches such as FL-WBC. In particular, the final backdoor accuracy drops dramatically (e.g., 11.6% IID and 2.7% non-IID), demonstrating effective mitigation of lingering poisoning effects. The group also analyzed the influence of the regularization rate, highlighting the robustness–accuracy trade-off.

We thank the LeadFL group for their precise reproduction of the algorithm and their comprehensive evaluation under both IID and non-IID settings.

B. DeComFL: Dimension-Free Communication via Zeroth-Order Optimization

[7] propose *DeComFL*, a communication-efficient federated learning framework that achieves communication complexity independent of model dimension.

1) Problem Motivation.: In standard federated learning (e.g., FedAvg), each client transmits a full gradient or model update vector of dimension d per round, resulting in $O(d)$ communication cost. For modern deep models, this becomes prohibitive. As highlighted in the original work, fine-tuning large foundation models can require hundreds of terabytes of data transfer.

2) Core Idea: Zeroth-Order Optimization.: DeComFL leverages zeroth-order (gradient-free) optimization using random gradient estimators (RGE). Instead of computing and transmitting full gradients, each client:

- Samples a shared random direction vector z generated from a public seed,
- Computes a scalar gradient estimate:

$$\hat{g} = \frac{f(x + \mu z) - f(x)}{\mu},$$

- Updates locally using $\hat{g} \cdot z$.

Crucially, clients transmit only:

- The random seed (or index),
- The scalar magnitude $\hat{g}$.

The server reconstructs the descent direction using the same seed. As a result, uplink communication becomes independent of model dimension.

3) Theoretical Guarantees.: The authors provide convergence analysis under smooth and non-convex objectives. Compared to FedAvg:

- FedAvg: $O(d)$ communication per round.
- DeComFL: $O(1)$ communication per client per perturbation.

Thus, communication cost becomes dimension-free while maintaining comparable convergence rates.

4) Original Experimental Results.: Experiments on MNIST, CIFAR-10, and ImageNet demonstrate:

- Over 99% communication reduction,
- Minor accuracy degradation (typically 1–2%),
- Improved performance using central RGE over forward RGE,
- Faster convergence with larger numbers of local steps K and perturbations P.

5) Class Implementation and Evaluation.: The DeComFL group implemented the zeroth-order communication protocol as described in Xu et al. Their implementation included:

- Shared random seed generation,
- Forward and central random gradient estimators (RGE),
- Scalar-only uplink communication,
- Server-side reconstruction of descent directions,
- Ablation over local update steps K and perturbation number P.

Experiments were conducted on CNN-MNIST. Table VI reports accuracy and uplink communication cost per round.

TABLE VI: DeComFL Implementation Results (CNN-MNIST)

Method	Accuracy (%)	Uplink Cost
FedAvg	94.22	198.70 MB
DeComFL (forward, $K = 1$)	30.09	421.875 KB
DeComFL (central, $K = 1$)	43.69	421.875 KB
DeComFL (forward, $K = 5$)	70.89	2.06 MB
DeComFL (central, $K = 5$)	91.97	2.06 MB
DeComFL (forward, $K = 10$)	83.95	4.12 MB
DeComFL (central, $K = 10$)	95.93	4.12 MB

The results confirm the core claim of DeComFL: communication can be reduced by several orders of magnitude compared to FedAvg while maintaining competitive accuracy when sufficient local steps are used. The central gradient estimator consistently outperforms the forward estimator. Additional experiments varying the perturbation number P further demonstrate the trade-off between computational overhead and accuracy.

We thank the DeComFL group for their faithful implementation and detailed ablation study, which clearly illustrates the scalability benefits of dimension-free communication in federated learning.

References

[1] Rflpa: Robust federated learning with privacy-preserving aggregation. *Advances in Neural Information Processing Systems*, 2024.

[2] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. *Advances in Neural Information Processing Systems*, pages 119–129, 2017.

[3] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. *Proceedings of the ACM Conference on Computer and Communications Security*, pages 1175–1191, 2017.

[4] Xiaoyu Cao, Minghong Fang, Jia Liu, and Neil Zhenqiang Gong. Fltrust: Byzantine-robust federated learning via trust bootstrapping. *Network and Distributed System Security Symposium (NDSS)*, 2021.

[5] J. Duan, J. Zhou, and Y. Li. Privacy-preserving distributed deep learning based on secret sharing. *Information Sciences*, 527:108–127, 2020.

[6] Clement Fung, Chris J. M. Yoon, and Ivan Beschastnikh. Mitigating sybils in federated learning poisoning. *Proceedings of the IEEE International Conference on Distributed Computing Systems*, 2018.

[7] Z. Li, B. Ying, Z. Liu, C. Dong, and H. Yang. Achieving dimension-free communication in federated learning via zeroth-order optimization. *International Conference on Learning Representations*, 2025.

[8] J. Park and H. Lim. Privacy-preserving federated learning using homomorphic encryption. *Applied Sciences*, 12(2):734, 2022.

[9] Chulin Zhu, Stefanie Roos, and L. Y. Chen. Leadfl: Client self-defense against model poisoning in federated learning. *Proceedings of the International Conference on Machine Learning*, pages 43158–43180, 2023.